

PRM393 – Mobile Programming

Lab 2: Journal Trend Analysis Mobile Application

Project Report

Student ID	SE180518
Subject	PRM393 – Mobile Programming
Assignment	Lab 2 – Journal Trend Analysis
Repository	PRM393_Lab2_SE180518
Date	June 2026

1. Introduction

For this lab, I built a Flutter mobile application called **Journal Trend Analyzer** that retrieves real scholarly publication data from OpenAlex and visualizes research trends for any topic the user searches.

Research publications are growing rapidly across many disciplines, making it increasingly important to identify emerging topics, influential papers, active researchers, and publication trends. Academic databases such as OpenAlex provide access to large-scale scholarly data that can be used for research analytics and decision support.

The app allows users to search for any research topic — such as Artificial Intelligence, Blockchain, or Cybersecurity — and instantly explore how that topic has evolved over time through interactive charts, rankings, and summary dashboards. All data is fetched live from the OpenAlex REST API without any backend or authentication required.

I chose OpenAlex as the data source because it is completely free, has no API key requirement for basic usage, covers over 250 million publications from global academic sources, and provides a clean and well-documented REST API that works well from a Flutter mobile client.

2. Learning Objectives

This assignment helped me practice and apply several key mobile development skills:

RESTful API Integration — I learned how to call an external REST API from a Flutter app, handle HTTP responses, parse complex JSON structures, and deal with real-world edge cases like missing fields and unusual data formats (OpenAlex stores abstracts as an inverted index, not plain text).

Asynchronous Programming — The app heavily uses `async/await` and `Future` to handle API calls without blocking the UI. I also implemented background data fetching so that analytics load asynchronously after the search results are already displayed.

State Management — I used the `Provider` package with `ChangeNotifier` to manage shared state across multiple screens. This was my first time using `Provider` in a real project and I found it much cleaner than passing data through constructors.

Data Visualization — I used the `fl_chart` library to build an interactive line chart showing publication trends over time. This required transforming raw API data into chart-compatible data points.

Clean Architecture — I organized the project into separate layers (models, services, state, screens, widgets) to keep the code maintainable and avoid mixing UI logic with business logic.

3. System Design

3.1 Project Structure

The project follows a clean layered architecture with clear separation of concerns between data, logic, and UI:

```
lib/
├── models/
│   ├── publication.dart    # Publication entity
│   └── trend_data.dart    # Analytics models
└── services/
```

```

|   ├── openalex_service.dart # API calls + analytics
|   └── history_service.dart  # Search history (SharedPreferences)
└── state/
    ├── search_provider.dart  # ChangeNotifier state management
└── screens/
    ├── splash_screen.dart
    ├── search_screen.dart
    ├── publication_detail_screen.dart
    ├── trend_screen.dart
    └── dashboard_screen.dart
└── widgets/
    ├── loading_widget.dart   # LoadingList, EmptyState, ErrorState
└── main.dart

```

Models layer — Pure Dart data classes. Publication represents a single scholarly work. trend_data.dart contains lightweight models for analytics results.

Services layer — OpenAlexService handles all HTTP communication and analytics computation. HistoryService handles local persistence of search history.

State layer — SearchProvider is the single source of truth for all app state, managing the search lifecycle, pagination, background analytics, and search history.

Screens layer — Each screen focuses purely on presentation, reading from SearchProvider but never calling API methods directly.

3.2 State Management with Provider

The application uses the Provider pattern with ChangeNotifier. When a user searches, the provider: sets state to loading → fetches first 50 results → shows results immediately → starts background fetch of up to 200 publications → computes analytics → notifies listeners to update Trend and Dashboard tabs.

3.3 OpenAlex API Integration

Base URL: <https://api.openalex.org> **Endpoint:** GET /works

Parameter	Value	Purpose
search	user's query	Full-text search across titles and abstracts
sort	cited_by_count:desc	Sort by most cited first
per_page	50	Results per page
page	1, 2, 3...	Pagination support

mailto	se180518@fpt.edu.vn	Polite pool for higher rate limits
--------	---------------------	------------------------------------

One interesting technical challenge was the abstract format. OpenAlex stores abstracts as an inverted index (word → list of positions) rather than plain text. The app reconstructs the original text by reversing this mapping — sorting all word-position pairs by position and joining the words back into a sentence.

4. Functional Requirements Implementation

4.1 Topic Search – Search Screen

The search screen is the entry point of the app. Users can type any research keyword and get results sorted by citation count. Features include:

- Search bar with keyboard submit action
- 9 popular topic chips for quick one-tap search (AI, Machine Learning, Blockchain, IoT, Cybersecurity, Data Science, Software Engineering, Deep Learning, NLP)
- Results list showing title, authors (first 2 + "et al."), year, citation count, and journal name
- Rank badges — top 3 results highlighted with gold badge
- Infinite scroll pagination — automatically loads next 50 results when scrolling to bottom
- Search history — last 10 searches saved locally, accessible via history icon
- Shimmer loading animation while fetching
- Error state with retry button if API call fails

4.2 Publication Details – Publication Detail Screen

Tapping any publication card opens a full detail screen showing:

- Full publication title
- Citation count and publication year (stat chips)
- Journal/source name
- DOI (Digital Object Identifier)
- Author list — up to 5 authors shown, remaining count displayed
- Abstract — reconstructed from OpenAlex inverted index format
- "Open Full Paper" button — opens the paper URL in device browser

4.3 Publication Trend Analysis – Trend Screen

The trend screen provides 4 sub-sections accessible via chip tabs:

Publication Trend Chart — Line chart (fl_chart) showing number of publications per year from 1990 to present. Smooth curved line with filled area below. X-axis shows years, Y-axis shows publication count.

Top Influential Papers — Top 20 papers ranked by citation count (highest to lowest). Shows title, citation count, and year. Tap to view publication details.

Top Research Journals — Top 10 journals ranked by number of publications, with horizontal progress bar showing relative contribution.

Top Contributing Authors — Top 10 authors ranked by number of papers, with progress bar visualization.

4.7 Research Trend Dashboard – Dashboard Screen

Summary dashboard with key insights about the searched topic:

Metric	Description
Total Publications	Total number of publications analyzed
Avg Citations	Average citation count across all publications
Most Active Year	Year with the highest publication count
Max Citations	Highest single-paper citation count
Top Journal	Journal with most publications on the topic
Top Author	Author with most publications on the topic
Most Influential Paper	Paper with highest citation count

5. Technical Stack

Package	Version	Purpose
flutter	3.44.0	UI framework
provider	6.1.5	State management
http	1.6.0	HTTP client for API calls
fl_chart	0.69.2	Line chart visualization
shared_preferences	2.5.5	Local search history storage

shimmer	3.0.0	Loading skeleton animation
url_launcher	6.3.2	Open paper URLs in browser

Android Configuration: minSdk 21 (Android 5.0), targetSdk 34 (Android 14), Internet permission added in AndroidManifest.xml.

6. AI-Assisted Code Review

6.1 Tool Used

CodeRabbit (coderabbit.ai) — an AI-powered automated code review tool that analyzes code for bugs, security issues, performance problems, and code quality improvements.

6.2 Issues Found and Fixed

Issue 1: Silent error handling in analytics fetch

Location: search_provider.dart — `_fetchAllForAnalytics()` method

Problem: The catch block was completely empty. Any exception during the background analytics fetch would be silently swallowed. If the analytics fetch failed, the Trend and Dashboard tabs would show a loading indicator forever with no way for the user to know something went wrong.

Fix applied: Added proper EmptyState and ErrorState widgets on the Trend and Dashboard screens so users see meaningful feedback instead of an indefinite loading state.

Issue 2: Potential notifyListeners() after dispose

Location: search_provider.dart — async methods

Problem: notifyListeners() was being called inside async functions that could complete after the widget tree is disposed. While Flutter's ChangeNotifier handles this internally, it's considered best practice to guard against it.

Fix applied: Acknowledged as low risk since SearchProvider lives for the full app lifecycle. No structural change needed.

Issue 3: Hardcoded email address in service

Location: openalex_service.dart — `_email` constant

Problem: The email used for OpenAlex's polite pool is hardcoded in source code. In production, configuration values should be externalized.

Fix applied: Acceptable for lab scope. In production, this would be moved to a .env file or Flutter's --dart-define build configuration.

6.3 Code Improvements Applied

- Replaced basic `CircularProgressIndicator` with shimmer skeleton loading for better perceived performance
- Added dedicated `EmptyState` widget with descriptive messages for all empty data scenarios
- Added `ErrorState` widget with a `Retry` button so users can recover from failed API calls
- Set a 15-second timeout on all HTTP requests to prevent indefinite hanging
- Used `IndexedStack` for bottom navigation so each tab preserves its scroll position and state when switching

7. User Interface Design

7.1 Screen Navigation Flow

```
SplashScreen (2s animated intro)
└─ HomeShell (BottomNavigationBar)
    ├── Tab 0: SearchScreen
    │   └─ PublicationDetailScreen
    ├── Tab 1: TrendScreen
    │   ├── Publication Trend Chart
    │   ├── Top Papers list
    │   ├── Top Journals list
    │   └─ Top Authors list
    └─ Tab 2: DashboardScreen
```

7.2 Design Decisions

Material 3 with Indigo color scheme — Indigo conveys professionalism and academic credibility, appropriate for a research tool. Material 3 provides modern, accessible UI components.

Bottom Navigation Bar with `IndexedStack` — Standard mobile navigation pattern. Using `IndexedStack` ensures each tab's state is preserved when switching — search results don't reset when the user goes to Trend tab and comes back.

Shimmer loading — Instead of a simple spinner, shimmer skeleton screens give users a preview of the layout while data loads, making the app feel faster and more polished.

Rank badges on search results — Top 3 results get a gold badge to draw attention to the most highly cited papers, helping users quickly identify influential work.

Progress bars for rankings — For journals and authors, horizontal progress bars make it visually obvious how dominant the top entry is compared to others — more intuitive than plain numbers.

8. Challenges and Lessons Learned

8.1 Challenges Faced

Challenge 1: Abstract inverted index format

When I first tested the API, I expected abstracts to be plain text strings. Instead, OpenAlex returns them as a dictionary mapping each word to a list of its position indices. I had to write a reconstruction algorithm that reverses this format — collecting all word-position pairs, sorting by position, and joining the words back into a sentence.

Challenge 2: Two-stage data loading

The app needs fast search results AND comprehensive analytics. Fetching all 200 publications at once before showing anything would feel too slow. My solution was to show the first 50 results immediately, then fetch more in the background for analytics. The tricky part was coordinating the loading states across the Trend and Dashboard tabs.

Challenge 3: fl_chart configuration

Getting the line chart to look correct with proper axis labels required reading through the fl_chart documentation carefully. OpenAlex data can span many years, so the x-axis needed dynamic interval calculation to avoid overlapping labels.

Challenge 4: Pagination state management

When implementing pagination, I had to be careful not to mix the paginated _results list (for the search tab) with _allPubs (for analytics). These serve different purposes and needed to be managed independently in SearchProvider.

8.2 Lessons Learned

- **Read API documentation before coding** — Wasted time on the abstract field because I assumed it would be plain text. Reading docs first would have saved that time.
- **Design state before writing code** — The two-level data model needed upfront thought. Good state structure design made background fetching much easier to

implement correctly.

- **Provider is clean but requires discipline** — Provider works well for sharing state but keeping the provider lean is important for maintainability.
- **Background async is powerful but tricky** — Running analytics in the background improved UX significantly, but required careful thought about state reset scenarios.

9. Future Improvements

Given more time, several enhancements would make the app more useful:

- Filter by year range — let users restrict results to a specific time period
- Bar chart option — alternative visualization for the publication trend data
- Save/bookmark publications — let users save interesting papers to a local reading list
- Export trend data — share the trend chart as an image or export data as CSV
- Compare topics — show two topics' trend lines on the same chart for side-by-side comparison

10. Conclusion

The Journal Trend Analyzer successfully meets all the functional and technical requirements of Lab 2. The app is built with Flutter and Dart, consumes the OpenAlex REST API directly from the mobile client without any backend, handles JSON data processing and asynchronous state management, visualizes data through interactive charts, and runs on Android.

The project helped me gain hands-on experience with API integration, Provider state management, data visualization with `fl_chart`, and organizing a Flutter project with a clean layered architecture. The AI-assisted code review with CodeRabbit caught real issues that I was able to address before submission.

The most challenging and interesting part of this lab was designing the two-stage loading system — showing fast initial results while computing deeper analytics in the background. This kind of UX consideration makes the difference between an app that feels slow and one that feels responsive.

References

1. OpenAlex API Documentation — <https://developers.openalex.org>
2. OpenAlex API Reference: Works — <https://developers.openalex.org/api-reference/works>
3. Flutter Documentation — <https://flutter.dev/docs>
4. Provider Package — <https://pub.dev/packages/provider>
5. fl_chart Package — https://pub.dev/packages/fl_chart
6. shared_preferences Package — https://pub.dev/packages/shared_preferences
7. CodeRabbit AI Code Review — <https://coderabbit.ai>
8. Material Design 3 — <https://m3.material.io>